SIMULADO P1
SISTEMAS DISTRIBUÍDOS

Caderno de questões • Gabarito comentado completo • Folha de pontuação ao final

20 questões discursivas | 100 pontos | Dificuldade alta | Tempo sugerido: 4 horas

Orientações

Este simulado foi reorganizado para privilegiar os conteúdos estudados: Chord, relógios vetoriais, estimativas de escala, arquiteturas cliente-servidor, broker/wrapper e ordenação de mensagens. A antiga questão sobre relógios escalares de Lamport foi retirada.

Nas questões de tabelas, “envia” é evento de transmissão e “entrega” é disponibilização da mensagem à aplicação. As mensagens são multicast ao grupo P0, P1 e P2, salvo indicação contrária. Assuma que as entregas explicitadas ocorreram; não invente eventos ausentes.

Mostre contas e unidades, justifique relações de causalidade com caminhos ou vetores e indique claramente quais propriedades de entrega são satisfeitas. Não confunda FIFO por remetente, causalidade e ordem total.

Parte I — Fundamentos, arquitetura e dimensionamento

Questão 01 — Conceito, transparência e abertura [5 pontos]

A. Defina sistema distribuído com autonomia e visão única/coerente. Diferencie computadores conectados de componentes cooperantes.

B. Discuta transparências de acesso, localização, replicação e falha, dando um exemplo e uma limitação prática para cada uma.

C. Explique portabilidade, interoperabilidade e extensibilidade. Por que uma interface padronizada não elimina latência nem falhas de rede?

Questão 02 — Estimativa de plataforma social (estilo Twitter) [7 pontos]

Considere 80 milhões de usuários ativos por dia, 3 publicações/usuário/dia, 400 bytes de texto+metadados por publicação; 35% contêm mídia de 1,8 MB. Pico de ingestão = 8× a média; replicação = 3; retenção = 5 anos; crescimento de publicações = 20% ao ano. Considere 1 MB = 10^6 bytes e ano = 365 dias.

A. Calcule publicações por dia, média e pico de publicações/s.

B. Calcule volume lógico diário de texto/metadados e de mídia, separadamente.

C. Estime armazenamento físico diário com replicação e capacidade física acumulada em cinco anos, considerando crescimento composto. Declare como tratou o primeiro ano.

D. Proponha sharding, cache e filas de ingestão. Explique duas limitações e um possível ponto único de falha.

Questão 03 — Modelos temporais e comunicação [4 pontos]

A. Diferencie modelos síncrono, assíncrono e parcialmente síncrono em termos de limites de atraso e processamento.

B. Uma RPC expira em 100 ms após 350 ms de latência. Isso prova que o servidor falhou? Justifique.

C. Diferencie modelo de sistema assíncrono de comunicação RPC assíncrona.

Questão 04 — Broker versus wrapper [6 pontos]

Uma organização integra ERP legado SOAP, API REST e mensageria AMQP.

A. Desenhe uma solução baseada em wrappers e outra usando broker, identificando as responsabilidades de cada elemento.

B. Compare tradução de interfaces/protocolos, descoberta, acoplamento temporal e acoplamento de referência.

C. Explique por que broker e wrapper podem coexistir e quais falhas operacionais cada um introduz.

Questão 05 — Onde o código é executado? Cliente-servidor [6 pontos]

Uma loja virtual usa navegador com JavaScript, SSR no backend, banco PostgreSQL, CDN, API de pagamentos e serviço de frete.

A. Classifique o local de execução de: validação de formulário, cálculo do preço final, consulta SQL, renderização SSR, cache CDN e cálculo de frete remoto.

B. Compare thin client, rich client e arquitetura de três camadas. Camada lógica implica máquina física distinta?

C. Um atacante modifica o JavaScript local para alterar o preço. Onde deve estar a validação definitiva? Justifique.

Questão 06 — P2P, CDN e arquitetura híbrida [4 pontos]

A. Compare P2P estruturado, não estruturado e híbrido em busca, gerenciamento e tolerância a falhas.

B. Descreva tracker, peers e transferência de partes no BitTorrent.

C. Explique quando uma CDN ou edge pode reduzir latência sem eliminar a necessidade do servidor de origem.

Parte II — Relógios vetoriais e multicast

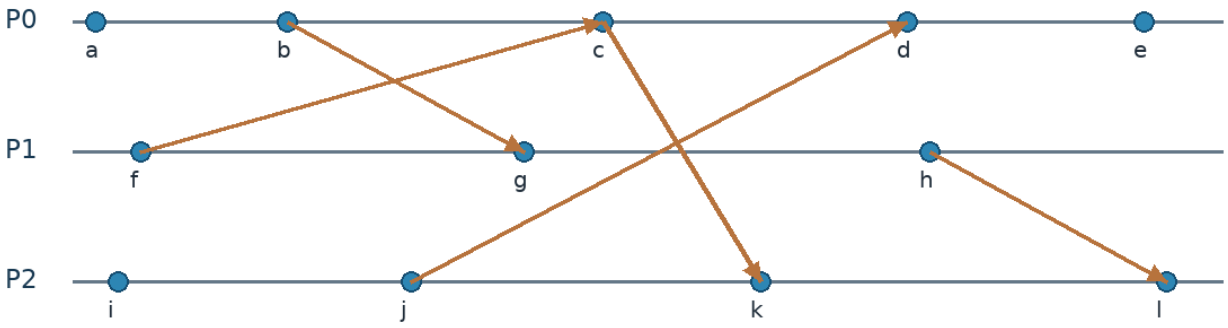
Questão 07 — Relógios vetoriais em diagrama [8 pontos]

Todos os vetores começam em (0,0,0). Cada evento incrementa a componente local. Em recepção, faça máximo componente a componente e incremente o receptor.

A. Calcule V(a), V(b), ..., V(l) em ordem alfabética.

B. Identifique três pares causalmente ordenados e dois pares concorrentes, justificando.

C. Os vetores estabelecem uma ordem total? Apresente contraexemplo usando eventos do desenho.



Setas entre processos: b→g, f→c, j→d, c→k e h→l. A ordem dentro de cada processo é da esquerda para a direita.

Questão 08 — Tabela I — FIFO, causal e ordem total [6 pontos]

P1 envia m2 somente após entregar m1. Todos os processos participam do grupo multicast; considere que P0 também entrega a própria m1, conforme registrado na tabela.

Tempo	P0	P1	P2
1	Envia m1		
2		Entrega m1	
3		Envia m2	Entrega m2
4	Entrega m2		Entrega m1
5	Entrega m1	Entrega m2	

A. Converta a tabela em três linhas de tempo, marcando envios, entregas e relações de causa e efeito.

B. A execução respeita FIFO por remetente? Causal? Ordem total? Justifique cada resposta sem confundir os conceitos.

C. Proponha alterações mínimas nas entregas para respeitar causalidade e ordem total simultaneamente.

Questão 09 — Tabela II — Ordem total sem causalidade entre envios [6 pontos]

Os envios de m1 e m2 são concorrentes; cada remetente envia somente uma mensagem.

Tempo	P0	P1	P2
1	Envia m1	Envia m2	
2	Entrega m1	Entrega m2	Entrega m1
3	Entrega m2	Entrega m1	Entrega m2

A. Desenhe o diagrama de eventos e setas de transmissão.

B. Julgue FIFO, causal e ordem total. Explique por que não existe dependência causal entre os envios.

C. Mostre duas sequências globais diferentes que seriam permitidas por ordem total e explique o que os processos precisam compartilhar.

Questão 10 — Tabela III — Três mensagens e dependências cruzadas [6 pontos]

Em P0, o envio de m3 ocorre depois do envio de m1. Em P1, o envio de m2 ocorre depois da entrega de m1. Não há dependência causal conhecida entre m2 e m3.

Tempo	P0	P1	P2
1	Envia m1		
2		Entrega m1	Entrega m1
3	Envia m3	Envia m2	
4	Entrega m2	Entrega m2	Entrega m3
5	Entrega m3	Entrega m3	Entrega m2

A. Construa o diagrama de eventos. Escreva todas as dependências causais entre os envios.

B. Avalie FIFO, causal e total; indique o primeiro ponto de divergência, se existir.

C. P2 pode entregar m3 antes de m2 em multicast causal? E em multicast total, se P0 e P1 entregaram m2 antes de m3?

D. Escreva uma sequência de entregas que respeite simultaneamente causal e total.

Questão 11 — RPC/RMI e semântica de falhas [4 pontos]

A. Descreva a chamada RPC completa: stub cliente, marshaling, rede, stub servidor, execução, resposta e unmarshaling.

B. Uma operação debitar(100) executa no servidor, mas a resposta se perde. Por que retry pode duplicar efeito?

C. Compare at-least-once, at-most-once e idempotência. Explique por que RMI não elimina o problema.

Parte III — Chord, naming e disseminação

Questão 12 — Chord: finger table e busca de 49 para 15 [8 pontos]

Anel autoral com m=6 (IDs 0 a 63) e nós ativos: 1, 4, 8, 11, 15, 19, 22, 26, 29, 33, 37, 41, 45, 49, 53, 58, 62. Esta distribuição é uma adaptação didática, não uma reprodução do diagrama original da lista do professor.

A. Determine succ(15), succ(16), succ(48) e succ(63).

B. Calcule as seis entradas da finger table do nó 49, mostrando cada start=(49+2^(i−1)) mod 64.

C. Simule a busca de ID 15 a partir do nó 49 usando closest preceding finger. Registre todos os saltos.

D. Compare com a rota que usa apenas sucessores; calcule quantos saltos foram poupados.

Questão 13 — Chord: mudanças no anel [5 pontos]

Partindo do anel da questão 12, entra o nó 13 e depois o nó 19 falha.

A. Indique quais intervalos de chaves mudam de responsável em cada evento.

B. Explique o papel de sucessor, predecessor, stabilize, notify e fix_fingers.

C. A chave continuar localizável garante que o valor permaneça disponível? Discuta replicação.

Questão 14 — Chord: pseudocódigo assíncrono [5 pontos]

Escreva pseudocódigo orientado a eventos para FIND(key, requestId, origin) e FOUND(requestId, successor), sem chamadas RPC bloqueantes.

A. Indique o estado mantido por cada nó e a condição de responsabilidade.

B. Use closestPrecedingNode para encaminhar consultas.

C. Descreva timeout, identificador de correlação e uma estratégia contra respostas tardias ou duplicadas.

Questão 15 — Naming, mobilidade e identificadores [4 pontos]

A. Defina nome, identificador, endereço e ponto de acesso; explique as três propriedades formais de um identificador.

B. Um número de celular que pode ser reatribuído é identificador formal? Justifique.

C. Compare home-based, ponteiros de encaminhamento, broadcasting e flooding para encontrar uma entidade móvel.

Questão 16 — Broadcast, flooding e gossip [4 pontos]

Uma atualização deve alcançar 20 mil nós, distribuídos em vários domínios de rede.

A. Diferencie broadcast, multicast, flooding com TTL e gossip com fanout=3.

B. Explique alcance, duplicatas, sobrecarga e garantia probabilística.

C. Qual mecanismo pode alcançar nós fora de um único domínio de broadcast sem exigir envio direto a cada destino? Discuta as condições.

Questão 17 — Algoritmos epidêmicos e remoção de dados [3 pontos]

A réplica A exclui o registro X, enquanto B permanece offline e ainda o possui.

A. Explique por que anti-entropy pode ressuscitar X.

B. Descreva tombstones e condições para descarte seguro.

C. Diferencie anti-entropy de rumor mongering.

Parte IV — Integração e análise crítica

Questão 18 — Stateful, stateless e balanceamento [3 pontos]

A. Uma sessão fica somente na memória de uma instância, mas o balanceador distribui requisições aleatoriamente. O que falha?

B. Compare sticky sessions, sessão em Redis e tokens verificáveis sem sessão local.

C. Discuta recuperação após falha, escalabilidade e consistência do estado.

Questão 19 — Replicação, consistência e latência geográfica [3 pontos]

Uma plataforma atende São Paulo e Frankfurt e aceita escritas locais durante uma partição.

A. Descreva uma execução que gere leituras divergentes.

B. Compare exigir confirmação síncrona de ambas as regiões com aceitar atualizações locais e reconciliar depois.

C. Explique por que replicação/cache pode melhorar desempenho e, simultaneamente, criar problemas de consistência.

Questão 20 — Projeto integrador: revisão de arquitetura [3 pontos]

Uma API global tem p95 de 3 s, falha quando o broker cai, depende de um ERP legado e atende dispositivos móveis intermitentes.

A. Desenhe uma proposta com camadas lógicas, broker e/ou wrapper, cache, descoberta de serviços e processamento assíncrono.

B. Justifique cinco decisões técnicas vinculadas a desempenho, disponibilidade, escalabilidade e abertura.

C. Identifique duas novas falhas ou compromissos criados pela sua própria arquitetura.

GABARITO COMENTADO — ETAPA 2

Resolução integral das 20 questões, com raciocínio, contas intermediárias, tabelas, exemplos de respostas discursivas e critérios de pontuação.

O caderno de questões está integralmente antes deste gabarito. Este gabarito adota explicitamente as convenções declaradas nos enunciados. Quando a tabela é incompleta, diferencia “não foi mostrado” de “não ocorreu”.

Notação: x → y significa happened-before; x || y significa concorrência. V(x)=(v0,v1,v2). No Chord, o intervalo (a,b] exclui a e inclui b, percorrendo o anel no sentido horário.

PARTE I — Fundamentos, arquitetura e dimensionamento

Questão 01 — Conceito, transparência e abertura [5 pontos]

A. Conceito formal

Um sistema distribuído é um conjunto de elementos computacionais autônomos, interconectados e cooperantes, que oferece a seus usuários a percepção de um sistema único e coerente. Autonomia significa que cada elemento dispõe de processamento/estado próprios; coerência exige integração de serviços e regras de interação. Computadores que apenas trocam arquivos não formam, necessariamente, uma aplicação distribuída integrada.

B. Quatro transparências e seus limites

Transparência	Exemplo	Limite / custo
Acesso		
	A mesma API consulta recurso local ou remoto.	
		Chamadas remotas têm latência, timeout e falhas diferentes.
Localização		
	Nome lógico permanece igual após mover serviço.	
		Descoberta/atualização de endereço pode ficar desatualizada.
Replicação		
	Cliente vê um serviço apesar de múltiplas réplicas.	
		Conflitos, invalidação e consistência podem aparecer.

Falha		
	Retry/failover mascara queda de instância.	
		Não se pode mascarar toda falha; retry pode duplicar efeitos.

C. Abertura

Interoperabilidade: componentes heterogêneos conseguem trocar informações e cooperar seguindo protocolos e formatos acordados. Portabilidade: uma implementação pode executar em diferentes plataformas com pouca ou nenhuma adaptação. Extensibilidade: é possível adicionar componentes/funcionalidades sem redesenhar todo o sistema. Interfaces bem definidas especificam operações, tipos e semântica; não tornam a rede instantânea ou infalível.

Critério de correção (5 pontos)

definição e distinção: 1.5 p; quatro transparências com limites: 2 p; três conceitos de abertura e rede: 1.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 02 — Estimativa de plataforma social [7 pontos]

A. Vazão

Publicações/dia = 80.000.000 × 3 = 240.000.000. Média = 240.000.000 / 86.400 = 2.777,78 posts/s. Pico = 8 × 2.777,78 = 22.222,22 posts/s. O pico representa ingestão de posts, não requisições totais de leitura, curtidas ou comentários.

B. Armazenamento lógico diário

Texto + metadados = 240.000.000 × 400 B = 96.000.000.000 B = 96 GB = 0,096 TB. Posts com mídia = 240.000.000 × 0,35 = 84.000.000. Mídia = 84.000.000 × 1,8 MB = 151.200.000 MB = 151.200 GB = 151,2 TB. Total lógico = 151,296 TB/dia.

C. Replicação e crescimento

Volume físico inicial por dia = 151,296 × 3 = 453,888 TB/dia. O ano 1 mantém a taxa inicial; nos anos 2 a 5 a taxa cresce 20% ao ano. Soma geométrica = 1 + 1,2 + 1,44 + 1,728 + 2,0736 = 7,4416.

Capacidade física acumulada em cinco anos = 453,888 TB/dia × 365 dias/ano × 7,4416 = 1.232.843,323392 TB ≈ 1.232,84 PB ≈ 1,233 EB (unidades decimais).

Ano	Multiplicador	Volume físico do ano (TB)
1		
	1.0	
		165,669.120
2		
	1.2	

		198,802.944
3		
	1.44	
		238,563.533
4		
	1.728	
		286,276.239
5		
	2.0736	
		343,531.487

Hipóteses: toda mídia tem tamanho médio fixo, todas as publicações são retidas, não há deduplicação/compressão, replicação 3× cobre conteúdo e metadados, e não contamos índices, backups extras, thumbnails, logs, capacidade ociosa ou erasure coding. Se a taxa cresce continuamente dentro de cada ano, o resultado muda.

D. Arquitetura e limites

Separar metadados e blobs; particionar posts por identificador/tempo ou chave de distribuição com atenção a hotspots; mídia em armazenamento de objetos; CDN para leitura; cache para timelines quentes; fila/stream para ingestão, fanout e processamento de mídia; consumidores idempotentes e replicação geográfica. Gargalos: fanout de celebridades, hot partitions, largura de banda de mídia, custo de retenção, consistência de contadores. SPOF potencial: broker único, coordenador de metadados ou serviço de autenticação sem redundância.

Critério de correção (7 pontos)

posts e taxas: 1.5 p; volumes lógicos: 1.5 p; replicação e projeção cinco anos: 2.5 p; arquitetura e limites: 1.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 03 — Modelos temporais e comunicação [4 pontos]

Síncrono: há limites conhecidos para tempo de execução, atraso de mensagem e/ou desvio de relógio. Assíncrono: não há limites temporais finitos conhecidos utilizáveis para decidir, em geral, se um nó está lento ou falhou. Parcialmente síncrono: limites existem mas podem ser desconhecidos, ou passam a valer após um período de estabilização.

Uma RPC que expira em 100 ms quando a rede leva 350 ms não prova falha do servidor. Timeout é suspeita, não diagnóstico conclusivo; o servidor pode estar processando, a rede pode atrasar ou a resposta pode ter sido perdida.

“Sistema assíncrono” é uma hipótese sobre limites de tempo; “RPC assíncrona” é um modelo de API que permite prosseguir sem bloquear aguardando resposta. Uma RPC assíncrona pode executar em ambiente parcialmente síncrono e vice-versa.

Critério de correção (4 pontos)

três modelos: 2 p; timeout: 1 p; duas acepções de assíncrono: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 04 — Broker versus wrapper [6 pontos]

A. Fluxos possíveis

Somente wrappers: Cliente → Wrapper REST/SOAP → ERP SOAP; Cliente → Wrapper REST → API REST; Cliente → Wrapper AMQP → produtor/consumidor AMQP. Cada wrapper adapta uma interface legada para contrato mais conveniente; o cliente ainda precisa localizar o destino ou usar um catálogo.

Com broker: Cliente → Broker/mediador → adaptador SOAP → ERP; Cliente → Broker → adaptador REST → API; Cliente → Broker → adaptador AMQP → mensageria. Broker pode oferecer roteamento, mediação e descoberta; para protocolos específicos, ainda usa wrappers/adaptadores.

B. Comparação

Critério	Wrapper	Broker
Papel		
	Adapta interface de um componente.	
		Media interação entre componentes.
Descoberta		
	Não necessariamente centraliza.	
		Pode manter registro/roteamento.
Referência		
	Cliente pode conhecer o endpoint do wrapper.	
		Cliente conhece broker ou endereço lógico.
Tempo		
	Pode continuar síncrono.	
		Pode ser síncrono ou assíncrono; broker não implica fila.
Falhas		
	Falha do adaptador isola acesso ao legado.	
		Broker sem redundância vira

		gargalo/SPOF.

C. Coexistência

Um broker não entende automaticamente semântica de operações SOAP/REST nem esquemas legados; wrappers traduzem formatos e contratos. Exemplo: broker roteia “ConsultarPedido” ao wrapper que converte o pedido para SOAP do ERP. Contrapartidas: mais saltos, observabilidade distribuída, governança de contratos e necessidade de redundância do broker.

Critério de correção (6 pontos)

dois fluxos: 2 p; quatro dimensões: 2.5 p; coexistência e falhas: 1.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 05 — Onde o código executa? [6 pontos]

Tarefa	Execução principal	Observação
Validação de formulário		
	Navegador (JavaScript)	
		Pode haver validação duplicada no backend.
Preço final		
	Backend de aplicação	
		Fonte de verdade; cliente não é confiável.
Consulta SQL		
	Motor PostgreSQL	
		Backend envia SQL, SGBD executa consulta.
Renderização SSR		
	Servidor de aplicação	
		Navegador recebe HTML e pode hidratar com JS.
Cache CDN		
	Nó edge/CDN	
		Cache não é necessariamente execução da lógica de negócio.
Cálculo de frete		
	Serviço remoto de frete	
		Backend consome API; execução

do cálculo ocorre no provedor.

Thin client: maior parcela de lógica/processamento no servidor; rich client: lógica relevante também no cliente, com maior autonomia e necessidade de atualização/segurança local. Três camadas lógicas: apresentação, aplicação e dados. Podem coexistir na mesma máquina ou ser distribuídas por várias; camada lógica ≠ máquina física.

A validação definitiva de preço, descontos, estoque e autorização deve ocorrer no backend confiável, com acesso a dados autoritativos; JavaScript pode ser modificado pelo usuário. Nunca confiar no preço enviado pelo cliente.

Critério de correção (6 pontos)

seis tarefas: 3 p; thin/rich/três camadas: 2 p; segurança e validação: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 06 — P2P, CDN e híbrida [4 pontos]

P2P estruturado: topologia e posicionamento de chaves seguem regras, como DHT/Chord; buscas tipicamente O(log N) sob hipóteses usuais; manutenção tem custo com churn. Não estruturado: peers conectados sem índice global rígido, busca por flooding, caminhadas aleatórias ou mecanismos locais; fácil adesão, mas busca pode gerar tráfego e cobertura incerta. Híbrido: algum componente central/superpeer apoia descoberta, enquanto conteúdo pode fluir diretamente entre peers.

BitTorrent: tracker (ou DHT) auxilia descoberta de peers; seeder possui conteúdo completo; leecher baixa e compartilha partes; seleção rarest-first prioriza peças raras; tit-for-tat favorece reciprocidade de upload, com variantes conforme cliente/protocolo.

CDN aproxima cópias dos consumidores e reduz RTT e carga da origem, mas misses, atualização, autenticação, conteúdo dinâmico e invalidação ainda podem exigir origem. CDN não elimina origem automaticamente.

Critério de correção (4 pontos)

tipos P2P: 2 p; BitTorrent: 1 p; CDN: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

PARTE II — Relógios vetoriais e multicast

Questão 07 — Relógios vetoriais [8 pontos]

A. Tabela completa

Evento	Processo	Vetor após evento	Justificativa curta
a			
	P0		
		(1,0,0)	
			local
b			
	P0		
		(2,0,0)	

			envia para g
c			
	P0		
		(3,1,0)	
			recebe f=(0,1,0); max com b e incrementa P0
d			
	P0		
		(4,1,2)	
			recebe j=(0,0,2); max com c e incrementa P0
e			
	P0		
		(5,1,2)	
			local
f			
	P1		
		(0,1,0)	
			envia para c
g			
	P1		
		(2,2,0)	
			recebe b=(2,0,0); max com f e incrementa P1
h			
	P1		
		(2,3,0)	
			envia para l
i			
	P2		
		(0,0,1)	
			local

j			
	P2		
		(0,0,2)	
			envia para d
k			
	P2		
		(3,1,3)	
			recebe c=(3,1,0); max com j e incrementa P2
l			
	P2		
		(3,3,4)	
			recebe h=(2,3,0); max com k e incrementa P2

Como calcular as recepções

c: max[(2,0,0),(0,1,0)] = (2,1,0) → (3,1,0). d: max[(3,1,0),(0,0,2)] = (3,1,2) → (4,1,2). g: max[(0,1,0),(2,0,0)] = (2,1,0) → (2,2,0). k: max[(0,0,2),(3,1,0)] = (3,1,2) → (3,1,3). l: max[(3,1,3),(2,3,0)] = (3,3,3) → (3,3,4).

B. Relações

Pares causais: b → g (mensagem), f → c (mensagem), j → d (mensagem), b → d (b→c→d), c → l (c→k→l). Pares concorrentes: a || f, pois (1,0,0) e (0,1,0) são incomparáveis; b || j, pois (2,0,0) e (0,0,2) são incomparáveis. Outras escolhas corretas são aceitas.

C. Ordem parcial, não total

V(x) < V(y) quando cada componente de V(x) ≤ a correspondente de V(y) e pelo menos uma é estritamente menor. Vetores incomparáveis indicam concorrência. Exemplo: V(a)=(1,0,0) e V(f)=(0,1,0). Nenhum vetor é ≤ ao outro; não há ordem causal entre a e f. Logo, vetores não fornecem ordem total por si sós.

Critério de correção (8 pontos)

12 vetores corretos: 5 p; pares e justificativas: 2 p; contraexemplo e definição: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 08 — Tabela I [6 pontos]

A. Transcrição em linhas de tempo

P0: t1 envia m1 → t4 entrega m2 → t5 entrega m1.

P1: t2 entrega m1 → t3 envia m2 → t5 entrega m2.

P2: t3 entrega m2 → t4 entrega m1.

Setas de transmissão: envio m1(P0,t1) → entregas m1(P1,t2), m1(P2,t4), m1(P0,t5); envio m2(P1,t3) → entregas m2(P2,t3), m2(P0,t4), m2(P1,t5). Entre P1 entregar m1 e enviar m2 existe aresta local; assim envio(m1) → envio(m2). A linha “tempo” é uma convenção de apresentação, não um relógio global necessário ao protocolo.

B. Propriedades

FIFO por remetente: não há violação demonstrada, pois P0 só envia m1 e P1 só envia m2. A condição FIFO não compara automaticamente mensagens de remetentes diferentes. Causal: violada em P0 e P2, que entregam m2 antes de m1, embora envio(m1) → envio(m2). Total: violada, pois P1 entrega m1,m2, enquanto P0 e P2 entregam m2,m1.

C. Correção

Uma correção de sequência de entregas é trocar as posições de m1 e m2 em P0 e P2, de modo que os três processos entreguem m1 antes de m2. A mudança pode exigir atrasar a entrega de m2 até m1 estar disponível. Preserva-se o envio de m2 após a entrega de m1 em P1. É uma correção lógica das sequências; tempos físicos específicos não são exigidos.

ATENÇÃO — No enunciado, P2 entrega m2 no mesmo tempo tabular em que P1 envia m2. Interprete os tempos como etapas com ordenação interna de transmissão, não como simultaneidade física estrita. Em um diagrama causal, o envio deve preceder a entrega.

Critério de correção (6 pontos)

linhas e dependência: 2 p; três classificações: 3 p; correção: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 09 — Tabela II [6 pontos]

P0: envia m1; entrega m1; entrega m2. P1: envia m2; entrega m2; entrega m1. P2: entrega m1; entrega m2. Setas: cada envio se liga às entregas correspondentes; envios são declarados concorrentes.

FIFO: respeitada de maneira trivial, pois cada remetente envia apenas uma mensagem. Causal: respeitada quanto à relação entre envios, pois m1 e m2 são concorrentes; nenhuma precedência causal obriga uma ordem de entrega entre elas. Total: violada, pois P1 vê m2,m1 e P0/P2 veem m1,m2.

Duas ordens globais possíveis: todos m1→m2, ou todos m2→m1. A propriedade exige que todos os processos que entreguem ambas concordem com a ordem relativa. Pode ser implementada com sequenciador, consenso/atomic broadcast ou outro protocolo apropriado; não basta comparar o horário de chegada.

Critério de correção (6 pontos)

diagrama: 2 p; três propriedades: 3 p; duas ordens e coordenação: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 10 — Tabela III [6 pontos]

A. Dependências entre envios

Envio(m1) → envio(m2), porque P1 entrega m1 antes de enviar m2. Envio(m1) → envio(m3), porque P0 envia m1 antes de m3. Não há caminho conhecido de envio(m2) até envio(m3) nem o inverso: m2 || m3. A tabela permite desenhar setas dos três envios às entregas registradas.

B. Julgamento criterioso

FIFO por remetente: não há inversão demonstrada. P0 envia m1 e m3, e P2 entrega m1 antes de m3; P1 também entrega m1 antes de m3. P0 não registra entrega de m1; logo não se pode testar o par de entregas m1/m3 em P0. Causal: P1 e P2 entregam m1 antes de m2/m3; nenhum deles viola as dependências. Em P0, m1 não aparece como entrega, então a tabela não basta para atestar causalidade completa se m1 deveria ser entregue localmente. Total: violada entre P0/P1 e P2 para m2,m3: P0/P1 entregam m2→m3, P2 entrega m3→m2.

C. Permissões

Em multicast causal, P2 pode entregar m3 antes de m2, porque os dois envios são concorrentes; basta ter entregue m1 antes de ambos. Em ordem total, isso não é permitido se P0 e P1 já entregaram m2 antes de m3.

D. Sequência válida

Uma ordem de entrega completa possível é m1→m2→m3 para P0, P1 e P2; outra é m1→m3→m2 para todos. Para realizar a primeira, P0 também deve registrar a entrega de m1, que está ausente da tabela; a correção do enunciado precisa adicionar esse evento se a exigência for entrega completa para todos os membros.

ATENÇÃO — O enunciado deliberadamente omite “P0 entrega m1”, apesar de P0 pertencer ao grupo. Não é correto inventar esse evento e afirmar que o traço original já comprova entrega causal em todos os processos. Além disso, P0 envia m3 antes de entregar m1: enviar não é entregar.

Critério de correção (6 pontos)

dependências: 1.5 p; FIFO/causal/total com ressalva: 2.5 p; resposta C: 1 p; sequência e completude: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 11 — RPC/RMI e falhas [4 pontos]

RPC: programa cliente invoca stub local → stub faz marshaling dos argumentos → camada de transporte envia requisição → stub servidor recebe e faz unmarshaling → rotina remota executa → stub servidor serializa resultado/erro → transporte retorna resposta → stub cliente desserializa e devolve ao chamador. O código de negócio executa no servidor, mesmo que a chamada pareça local.

Se debitar(100) foi persistido e a resposta se perdeu, o cliente não sabe se o débito ocorreu. Retry cego pode debitar mais R$ 100. At-least-once pode repetir execução; at-most-once evita execução duplicada para a mesma operação identificada, mas pode não garantir execução; exactly-once como efeito de negócio requer mecanismos adicionais e hipóteses explícitas.

Solução: requestId/chave de idempotência estável, persistida atomicamente com o resultado do débito; retries com a mesma chave retornam o resultado já armazenado. RMI muda o estilo de interface para métodos de objetos remotos, mas não elimina perda de resposta, timeout ou necessidade de idempotência.

Critério de correção (4 pontos)

fluxo e stubs: 1.5 p; cenário de perda: 1 p; semânticas e solução: 1.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

PARTE III — Chord, naming e disseminação

Questão 12 — Chord: nó 49 busca chave 15 [8 pontos]

A. Sucessores

IDs ativos ordenados: 1,4,8,11,15,19,22,26,29,33,37,41,45,49,53,58,62. succ(15)=15; succ(16)=19; succ(48)=49; succ(63)=1 (retorno circular).

B. Finger table de 49 (m=6)

i	start=(49+2^(i−1)) mod 64	finger[i]=succ(start)
1		
	50	
		53
2		
	51	
		53
3		
	53	
		53
4		
	57	
		58
5		
	1	
		1
6		
	17	
		19

C. Busca com closest preceding finger

Nó 49: succ(49)=53, e 15 não pertence a (49,53]. Entre os fingers no intervalo circular (49,15), o mais adiantado é 1 (finger[5]); encaminha 49→1. Nó 1: seus starts são 2,3,5,9,17,33, fingers 4,4,8,11,19,33; maior finger em (1,15) é 11. Encaminha 1→11. Nó 11: succ(11)=15; responde que o responsável é 15. Caminho de consulta/contato: 49→1→11→15. Contando enlaces até o responsável: 3 saltos.

D. Busca linear

Ponteiros sucessores: 49→53→58→62→1→4→8→11→15. Total: 8 saltos. Economia = 8−3 = 5 saltos, ou 62,5% em relação à rota linear desta consulta. A complexidade O(log N) é uma expectativa típica de Chord sob manutenção adequada, não a promessa de número fixo de saltos.

ATENÇÃO — A contagem pode variar em implementações que distinguem “encaminhamentos FIND” de “resposta com o sucessor”. A rubrica aceita 2 encaminhamentos FIND + identificação de 15, desde que o aluno explicite sua convenção.

Critério de correção (8 pontos)

sucessores: 1 p; seis fingers: 3 p; rota justificada: 2.5 p; linear e economia: 1.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 13 — Entrada e falha de nós [5 pontos]

Antes da entrada, 15 é responsável por (11,15] = {12,13,14,15}. Com a entrada de 13, o novo nó 13 assume (11,13] = {12,13}, e 15 fica com (13,15] = {14,15}.

Antes da falha, 19 é responsável por (15,19] = {16,17,18,19}. Após a falha, o sucessor ativo 22 passa a ser responsável por (15,22] = {16,17,18,19,20,21,22}, absorvendo o intervalo do nó 19. Isso descreve a atribuição de chaves, não a recuperação automática de seus valores.

successor aponta ao próximo nó ativo no anel; predecessor ao anterior. stabilize verifica se existe nó entre o atual e seu sucessor; notify informa um candidato a predecessor; fix_fingers atualiza atalhos de busca periodicamente. Para recuperar dados de um nó falho, é necessário que cópias, backups ou replicação já existam; o hashing consistente isoladamente não recupera bytes perdidos.

Critério de correção (5 pontos)

dois intervalos: 2 p; mecanismos: 2 p; localização versus disponibilidade: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 14 — Pseudocódigo assíncrono [5 pontos]

Convenções: id do nó = self; succ = sucessor conhecido; finger[1..m]; pending[requestId] apenas na origem; intervalo circular (a,b] trata wrap-around. send() não bloqueia. A busca supõe anel estabilizado; falhas exigem fallback.

```
on iniciarBusca(key):
    req = novoUUID()
    pending[req] = {key, prazo, estado:"aguardando"}
    send(self, FIND(key, req, self, hops=0))


on receber FIND(key, req, origin, hops):
    if key in (self, succ]:
        send(origin, FOUND(req, succ))
    else:
        next = closestPrecedingFinger(key)
        if next == self or next indisponível:
            next = succ
        send(next, FIND(key, req, origin, hops+1))


on receber FOUND(req, responsible):
    if req not in pending: return  # resposta tardia/duplicada
```

```
    if pending[req].estado != "aguardando": return
    pending[req].estado = "concluída"
    entregarResultado(req, responsible)
    remover pending[req]

on timeout(req):
    if req not in pending: return
    tentarNovamenteComMesmoReqOuNovoReq(req)
    # se desistir, marcar expirada e ignorar FOUND tardio
```

A condição (self,succ] identifica que succ é responsável. Para chave igual ao ID do próprio nó, o predecessor do nó anterior deve detectá-la e responder self como sucessor. Em uma implementação real, verifique limite de hops, sucessores alternativos, falhas e atualizações de membership. RequestId correlaciona respostas a chamadas simultâneas.

Critério de correção (5 pontos)

estado/condição: 1.5 p; handlers e roteamento: 2 p; timeout/correlação/duplicatas: 1.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 15 — Naming e mobilidade [4 pontos]

Nome: referência que denota entidade. Identificador: nome especial que identifica uma entidade de modo único no esquema, não designa duas entidades e não é reutilizado para outra entidade; a definição formal do livro também requer que cada entidade seja referida por no máximo um identificador dentro daquele esquema. Endereço: referência a um ponto de acesso. Ponto de acesso: local/interface pelo qual é possível contatar a entidade. Uma entidade pode ter vários endereços e migrar mantendo o identificador.

Número de celular reatribuível não é identificador permanente no sentido formal: após cancelamento pode referir-se a outra pessoa, violando não reutilização. Em determinado instante, ainda pode servir como nome ou referência de contato.

Técnica	Funcionamento	Trade-off
Home-based		
	Diretório na origem guarda endereço atual.	
		Consulta/atualização central e possível gargalo.
Forwarding pointers		
	Local antigo encaminha para novo.	
		Cadeias crescem; falhas intermediárias.

Broadcasting		
	Pergunta a todos no domínio de difusão.	
		Escopo local e tráfego de todos.
Flooding		
	Repassa a vizinhos com controle de duplicata/TTL.	
		Pode atravessar overlay, mas amplia mensagens.

Critério de correção (4 pontos)

conceitos e propriedades: 2 p; celular: 0.5 p; quatro mecanismos: 1.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 16 — Broadcasting, flooding e gossip [4 pontos]

Broadcast: uma transmissão alcança os participantes do domínio de difusão (p. ex. broadcast IPv4 em uma LAN), não necessariamente toda a Internet. Multicast: destinatários são membros de grupo específico. Flooding: cada nó encaminha a vizinhos elegíveis, descartando duplicatas e respeitando TTL. Gossip fanout=3: cada nó seleciona cerca de três contatos por rodada para difundir atualizações.

Flooding pode gerar múltiplas cópias do mesmo pacote e custo elevado; TTL limita alcance. Broadcast é limitado pelo domínio de difusão e por política dos roteadores. Gossip tende a distribuir carga, mas cobertura/tempo de convergência são probabilísticos; perda, churn e contatos repetidos podem atrasar ou impedir propagação.

Para 20 mil nós em vários domínios, flooding em overlay ou gossip com vizinhança/peer sampling interdomínios pode disseminar sem um emissor enviar individualmente a todos. Isso depende de conectividade, encaminhamento, TTL adequado e políticas de firewall; multicast roteado também pode fazê-lo quando suportado pela infraestrutura.

Critério de correção (4 pontos)

quatro definições: 2 p; comparação técnica: 1 p; mecanismo e condições: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 17 — Epidêmicos e remoção [3 pontos]

Sem histórico de exclusão, A não possui X e B possui X. Na anti-entropy, B pode oferecer X como “dado ausente” e A pode ressuscitá-lo. Tombstone é um marcador de exclusão com versão/causalidade que informa que X foi removido e prevalece sobre versões antigas.

Descartar tombstones exige garantia de que réplicas antigas não reaparecerão com o valor anterior: p. ex. confirmação de propagação a todos os participantes relevantes, janela de reparo limitada e protocolo de garbage collection seguro. TTL arbitrário pode ressuscitar dados após desconexão prolongada.

Anti-entropy: sincronização/reconciliação periódica de estados. Rumor mongering: propagação agressiva de novidades, reduzida ou encerrada quando deixam de ser novidade.

Critério de correção (3 pontos)

ressurreição: 1 p; tombstone e descarte: 1.5 p; duas estratégias: 0.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

PARTE IV — Integração e análise crítica

Questão 18 — Stateful, stateless e balanceamento [3 pontos]

Se sessão está apenas na instância A e a próxima requisição vai a B, B não reconhece autenticação/carrinho. Sticky sessions mantêm afinidade e são simples, mas prejudicam rebalanceamento e não preservam estado quando A cai. Redis centraliza estado de sessão e facilita escala horizontal, porém requer HA, TTL e política de consistência. Tokens assinados/verificáveis permitem autenticação sem sessão local, mas revogação, expiração e tamanho do token exigem cuidado; carrinho ainda pode precisar de armazenamento compartilhado.

Após crash, a sessão em memória se perde; com Redis replicado ou armazenamento durável, pode ser recuperada. Em ambiente distribuído, considerar leitura após escrita, invalidação, versionamento e políticas de expiração.

Critério de correção (3 pontos)

falha: 0.75 p; três opções: 1.5 p; recuperação/consistência: 0.75 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 19 — Replicação e consistência [3 pontos]

Durante partição, São Paulo aceita saldo=100→80 e Frankfurt aceita saldo=100→90 para o mesmo registro. Leituras locais divergem até reconciliação; dependendo da semântica, não basta escolher a última atualização sem perder uma operação.

Confirmar escrita nas duas regiões oferece coordenação mais forte, mas eleva latência e pode bloquear escrita quando a rede está particionada. Aceitar escritas locais aumenta disponibilidade e reduz latência, porém requer resolução de conflitos e admite consistência eventual/temporária.

Cache/replicação aproximam dados do consumidor e reduzem leituras remotas; em contrapartida, invalidar cópias e manter ordem de atualizações cria janelas de desatualização e conflitos. A escolha depende das invariantes de negócio.

Critério de correção (3 pontos)

exemplo de divergência: 1 p; duas políticas: 1 p; benefício e risco do cache: 1 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

Questão 20 — Projeto integrador [3 pontos]

A. Arquitetura proposta

Cliente móvel → CDN/API Gateway → serviços stateless (autenticação, catálogo, pedidos) → Redis/cache e banco de dados; pedidos assíncronos → broker replicado → workers idempotentes → wrapper do ERP legado. Serviço de descoberta/registro ou DNS interno resolve endpoints. Observabilidade distribuída mede p95 por etapa, filas e falhas. Dispositivos móveis mantêm fila local de operações e repetem com chaves de idempotência.

B. Cinco justificativas

1. CDN reduz RTT e carga em conteúdo cacheável. 2. Serviços stateless permitem balanceamento e escala horizontal. 3. Broker com múltiplas instâncias desacopla produtores/consumidores no tempo e absorve picos.

4. Wrapper isola o contrato legado do ERP. 5. Retries com idempotência tornam reconexões móveis mais seguras. Alternativas válidas: circuit breaker, backpressure, outbox transacional, replicação e discovery com health checks.

C. Novos riscos

Cache pode servir dados obsoletos; broker replicado pode ter lag, duplicatas e reordenação; outbox e idempotência aumentam complexidade; dependência de Redis/discovery pode criar novos SPOFs sem redundância. Uma resposta excelente explicita como mitigá-los e como medi-los.

Critério de correção (3 pontos)

arquitetura coerente: 1.25 p; cinco justificativas: 1.25 p; dois riscos: 0.5 p. Respostas equivalentes e tecnicamente justificadas recebem crédito integral.

REVISÃO FINAL — ERROS QUE MAIS TIRAM PONTOS

• Confundir “mensagem enviada” com “mensagem entregue”; causalidade entre envios não é a mesma coisa que ordem total de entregas.

• Chamar FIFO de “todos recebem na mesma ordem”: isso é ordem total, não FIFO por remetente.

• Usar posição horizontal de processos diferentes como relógio global; só linhas locais e setas geram happened-before.

• Somar componentes de vetores para comparar causalidade, em vez de comparar componente a componente.

• Esquecer módulo 2^m e wrap-around no Chord; ou confundir número de encaminhamentos com saltos até o responsável.

• Declarar que Chord recupera dados após falha sem replicação; hashing consistente resolve atribuição/localização.

• Confundir broker com fila obrigatoriamente assíncrona ou wrapper com roteador universal.

• Tratar timeout como prova de falha ou retry como operação sem risco de duplicidade.

• Ignorar unidades decimais/binárias e replicação ao dimensionar armazenamento.

• Inventar eventos ausentes em uma tabela de multicast e, com isso, afirmar garantias não demonstradas.

Resumo dos pontos por questão

Questão	Pontos
01	
	5
02	
	7
03	
	4
04	
	6
05	
	6
06	
	4
07	
	8
08	
	6

09	
	6
10	
	6
11	
	4
12	
	8
13	
	5
14	
	5
15	
	4
16	
	4
17	
	3
18	
	3
19	
	3
20	
	3
TOTAL	
	100

Material de estudo: respostas-modelo e rubricas sugeridas, não um gabarito oficial da disciplina. Nas questões 8–10, a interpretação distingue propriedades comprovadas pela execução das garantias de um protocolo em todas as execuções possíveis.

Folha de acompanhamento

Questão	Pontos	Sua nota
01	5	
02	7	
03	4	
04	6	
05	6	
06	4	
07	8	
08	6	
09	6	
10	6	
11	4	
12	8	
13	5	
14	5	
15	4	
16	4	
17	3	
18	3	
19	3	
20	3	